

# 本小节内容

队列的实战（通过链表实现）

## 队列的实战（通过链表实现）

在 113.5 小节我们已经讲解了队列的原理。

代码实战步骤依次是初始化队列，入队，出队，注意代码执行结束可以通过调试来观察队列内的元素，同时注意新建项目是 C++ 可执行程序，下面直接上代码：

```
#include <stdio.h>
#include <stdlib.h>

typedef int ElemType;
typedef struct LinkNode{
    ElemType data;
    struct LinkNode *next;
}LinkNode;
typedef struct{
    LinkNode *front,*rear;//链表头 链表尾
}LinkQueue;//先进先出

void InitQueue(LinkQueue &Q)
{
    Q.front=Q.rear=(LinkNode*)malloc(sizeof(LinkNode));//头和尾指向同一个结点
    Q.front->next=NULL;//头结点的 next 指针为 NULL
}

bool IsEmpty(LinkQueue Q)
{
    if(Q.front==Q.rear)
        return true;
    else
        return false;
}

//入队，尾部插入法
void EnQueue(LinkQueue &Q,ElemType x)
{
    LinkNode *s=(LinkNode *)malloc(sizeof(LinkNode));
    s->data=x;s->next=NULL;
    Q.rear->next=s;//rear 始终指向尾部
```

```

    Q.rear=s;
}
//出队 头部删除法
bool DeQueue(LinkQueue &Q,ElemType &x)
{
    if(Q.front==Q.rear) return false;//队列为空
    LinkNode *p=Q.front->next;//头结点什么都没存，所以头结点的下一个节点才有数据
    x=p->data;
    Q.front->next=p->next;//断链
    if(Q.rear==p)//删除的是最后一个元素
        Q.rear=Q.front;//队列置为空
    free(p);
    return true;
}
//《王道 C 督学营》课程
//头部删除法，尾部插入法
int main()
{
    LinkQueue Q;
    bool ret;
    ElemType element;//存储出队元素
    InitQueue(Q);//初始化队列
    EnQueue(Q,3);
    EnQueue(Q,4);
    EnQueue(Q,5);
    EnQueue(Q,6);
    EnQueue(Q,7);
    ret=DeQueue(Q,element);
    if(ret)
    {
        printf("出队成功,元素值为 %d\n",element);
    }else{
        printf("出队失败\n");
    }
    return 0;
}

```